

Antigravity로 배우는 에이전트 형 개발

🧑‍🚀 IDE 사용법보다 Agent에게 일을 맡기고 검증하는 운영 감각

🖥️ 화면 구조 · 🤖 Agent 작업 단위 · 🛡️ 모드/권한 · 📦 산출물 흐름 📌 실습 루프: 요청 → 계획 → 실행 → 검증

🧭 핵심 관점

코드 편집기 + 에이전트 매니저 + 브라우저가 하나의 작업 표면으로 연결

✍️ 학습 방식

문제 상황에서 출발해 실제 조작 흐름으로 확장

🧭 작업 방식 전환: 도구 소개 → 운영 감각



🧭 흐름: 문제 인식 → 🖥️ 화면 구조 → 🤖 Agent 운영 → 🛡️ 안전장치

🧭 아이콘 지도: 장표 읽는 법



개념

새 용어, 도구 범주, mental model 정리



조작

화면 이동, 버튼, 설정, 명령 입력 흐름



검증

diff, test, browser, evidence 기반 확인



안전

권한, 범위, review gate, 민감 정보 통제

🔄 반복 패턴: 🧩 이해 → 🖥️ 조작 → ✅ 확인 → 🛡️ 통제

🎯 학습 목표 3가지

1



작업 단위 전환

“코드 한 줄 수정”보다 “문제 해결 작업”
단위 위임 감각

2



화면 구조 이해

Editor, Manager, Browser의 역할
분리와 연결 구조

3



검증 루프 구성

Agent 답변보다 계획, diff, evidence,
review 기반 확인

🧭 Rules · Workflows · Skills · MCP = 기능 암기보다 Agent 운영 전략

🧱 코드 에이전트 도구: CLI vs IDE

📄 CLI형

🧵 터미널 기반 병렬 세션

Claude Code, Codex처럼 명령어 기반 탐색·수정·검증 반복



병렬 세션



텍스트 작업



리포지토리 탐색

💻 IDE형

💻 코드 에디터 기반 제어

Cursor, Antigravity처럼 편집기·파일·브라우저·작업 관리 통합



코드 수준 제어



UI 확인



작업 위임

VS

🖥️ CLI형: 터미널 기반 병렬 작업

01

🧵 세션 분리

탐색, 구현, 테스트, 리뷰를 별도 세션으로 분리

02

📄 텍스트 산출물

코드 조사, diff 작성, 로그 분석, 문서화에 적합

🕒 판단 기준: “어느 도구가 더 좋은가”보다 어떤 작업 표면이 필요한가

IDE형: 코드 확인 기반 작업 위임



Editor 중심

코드 위치, 파일 구조, 선택 영역, diff 직접 확인



UI 확인

웹앱과 브라우저 결과 동시 확인



작업 관리

Agent 계획과 산출물 검토

🚧 기존 AI IDE의 한계

🛠️ 기존 AI IDE

🧩 한 파일 안의 보조자

열린 코드, 선택 영역, 짧은 질문에는 강점. 긴 작업 흐름 유지에는 한계



🚀 AGENTIC IDE

🚀 계획과 수행의 실행 주체

📖 코드 읽기 · 🖥️ 터미널 실행 · 🌐 브라우저 확인 · 📦 산출물 작성 · ✅ 검토 요청

? 질문의 전환

○ 무슨 버튼?



🧩 어떤 작업 단위?

○ 버튼 중심

기능 암기 중심. 실제 프로젝트 적용 시점 판단 어려움

🧩 작업 단위 중심

문제, 제약, 검증 기준 중심. Agent 위임 단위 명확

Antigravity: 에이전트형 개발 플랫폼

 여러 Agent에게 작업을 맡기고 진행 상황과 결과를 관리하는 개발 환경

 IDE 기능 ·  Agent 실행 ·  브라우저 조작 ·  산출물 리뷰

 관점 전환: “AI가 붙은 에디터” → Agent 작업 운영 플랫폼



Agent Manager: 작업 지휘실



Agent Manager

Workspace A



기능 구현 Agent

Workspace B



버그 수정 Agent

Workspace C



리서치 Agent



역할: 코드 편집 화면이 아닌 Agent 작업 오케스트레이션 화면

● ⚡ ASYNCHRONOUS AGENTS

⚡ Asynchronous Agents: 비동기 작업 흐름

 **Agent 1**

로그인 버그 재현



 Evidence 작성

 **Agent 2**

UI 컴포넌트 수정



 Diff 준비

 **Agent 3**

문서 리서치



 요약 보고

 운영 포인트: 여러 작업 흐름의 우선순위와 승인 지점 관리

기본 AI IDE 기능: Agent · Tab · Command



Agent

큰 작업을 계획하고 수행하는 핵심
기능



Tab

코드 작성 중 강력한 자동완성



Command

편집기나 터미널에서 인라인 지시



중심 기능: Tab·Command보다 **Agent**, 나머지는 Agent 작업 보조 표면

Core Surfaces: Manager · Editor · Browser



Manager

작업을 맡기고 진행 상황과 산출물을
검토



Agentic Workspace



Editor

코드 수정, 파일 탐색,
Tab·Command 사용



Browser

웹앱, 대시보드, SCM, 리서치 화면
확인



Surface 1: Manager — 작업 생성과 검토



Agent Manager

여러 workspace와 Agent 대화를 작업
단위로 관리



시작

새 Agent 작업 생성



확인

진행 상태와 산출물 확인



검토

계획, diff, 보고서 피드백



Surface 2: Editor — 코드 수준 제어

📁 파일 탐색

프로젝트 구조와 변경 위치 확인

✍️ 직접 수정

필요 시 직접 개입

✨ AI 기능

Agent, Tab, Command 사용



Editor

Agent 변경을 가장 구체적으로 확인하는 화면

Surface 3: Browser — 웹 화면 기반 확인

 웹 UI 테스트

 대시보드 확인

 GitHub /
GitLab 작업

 브라우저 기반
개발

 리서치와 보고서

 Browser의 의미: 코드 밖 결과까지 Agent 작업 루프에 포함

기본 이동 1: Editor → Agent Manager

1

 Editor 상단 바의 Agent Manager 버튼

또는

2

 단축키 Cmd + E

 핵심 동선: 코드 확인 화면 → 작업 관리 화면

기본 이동 2: Manager → Editor



Workspace 선택

작업 중인 workspace
드롭다운 확인



Focus Editor

해당 workspace의
Editor로 포커스 이동



Open Editor

필요한 Editor 화면 열기



Agent Mental Model: 멀티스텝 실행 시스템



🧩 Agent 구성 요소 4가지

🧠 Reasoning Model

요청 이해와 계획·추론

🔧 Tools

브라우저, 터미널, 파일 작업 실행

📦 Artifacts

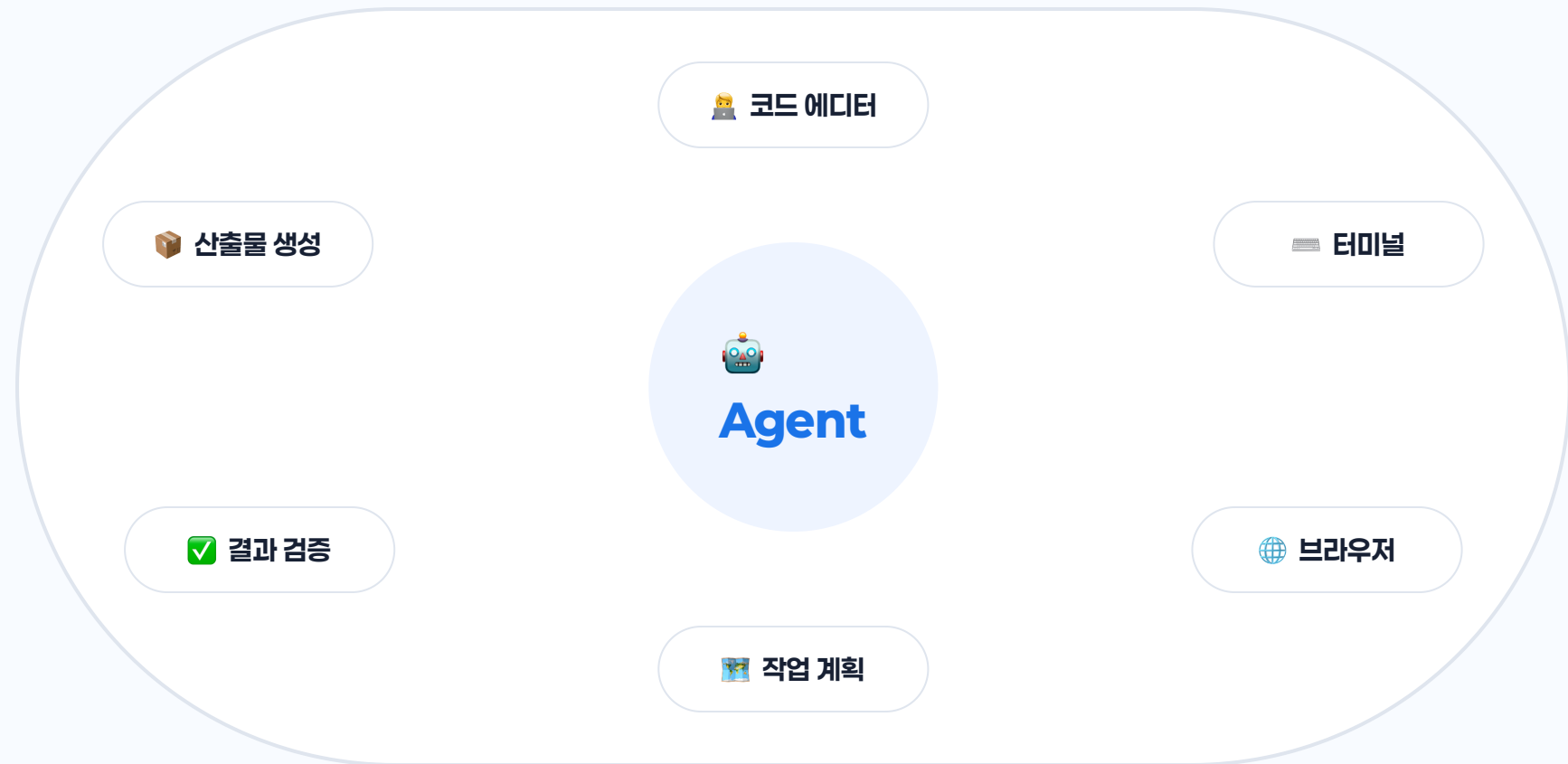
계획서, 결과물, 보고서 생성

📖 Knowledge

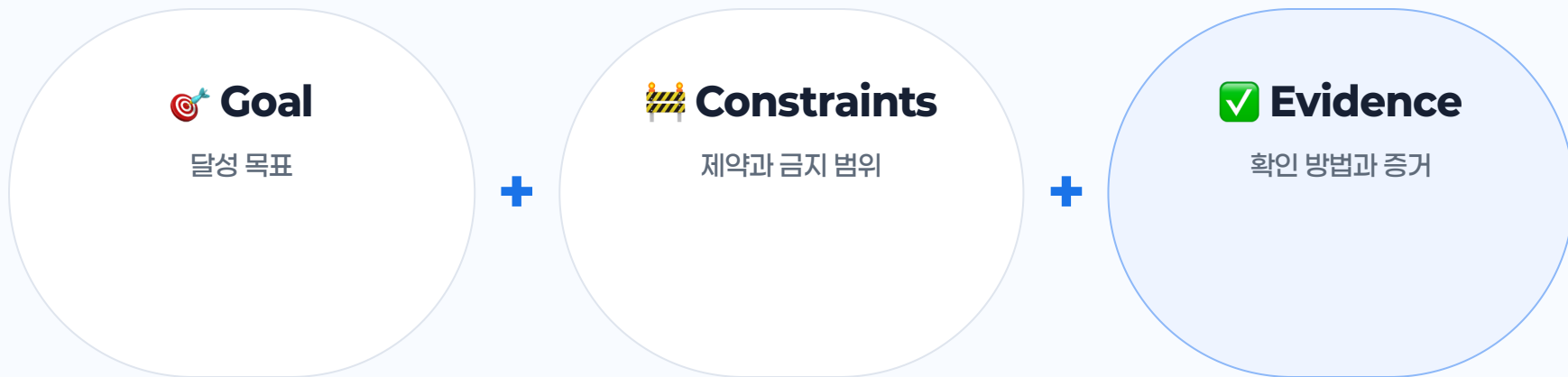
작업 문맥과 지식 참고

🧩 핵심 구분: 답변 생성이 아니라 추론 + 도구 + 산출물 + 지식 결합

Agent Toolkit: 사용 가능한 작업 도구

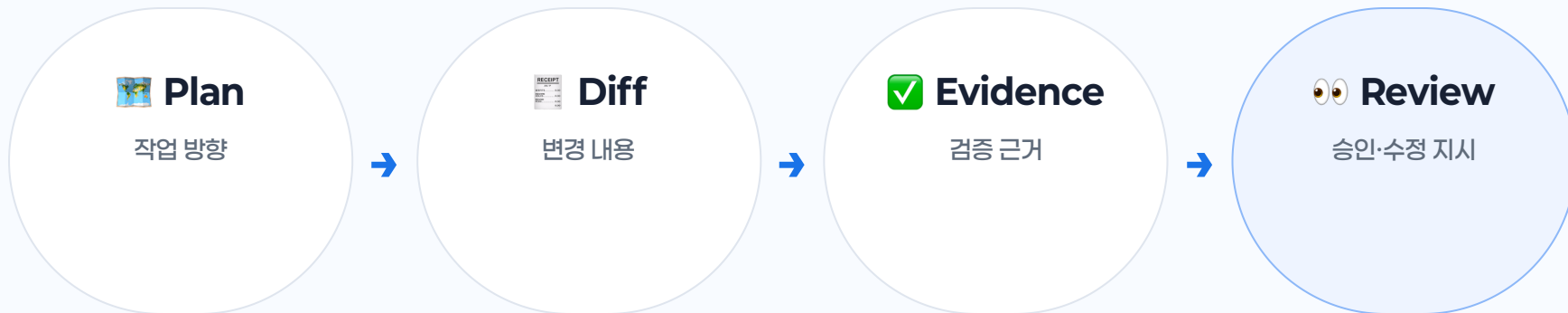


🖋️ Prompt Formula: Goal + Constraints + Evidence



✏️ 예시 구조: “재현 → 원인 후보 → 최소 수정 → 테스트 결과 보고”

📦 Artifacts: 작업 공유 화면



📦 Artifact Examples: 중간·최종 산출물



마크다운 문서



코드 diff



아키텍처 다이어그램



이미지



브라우저 녹화



작업 보고서



Artifact의 의미: Agent의 작업 방식과 결과를 확인하는 커뮤니케이션 수단

✓ Review Habit: 설명보다 근거

1. 🗺️ 계획 확인

문제 이해와 작업 범위 확인

2. 📄 변경 확인

diff 적용 파일과 변경 범위 확인

3. ✓ 검증 확인

테스트, 브라우저, 로그 근거 확인

⚙️ Agent Mode: Planning vs Fast

🕒 PLANNING MODE

🕒 조사와 계획이 필요한 작업

- 복잡한 기능 개발
- 깊은 리서치와 설계 판단
- 사용자 피드백이 필요한 작업

작업이
바뀌었어?

⚡ FAST MODE

⚡ 즉시 실행 가능한 작은 작업

- 변수명 변경
- 짧은 bash 명령
- 작은 범위의 코드 수정

🕒 Planning Mode: 조사 → 계획 → 검토

01

🔍 조사

코드베이스, 문서, 이슈 기반 맥락
수집

02

🗺️ 계획

Task Group과 Artifact 기반 작업
순서 정리

03

✅ 검토

방향 확인과 피드백 지점 확보

🕒 추천 상황: 복잡한 작업 · 품질 우선 작업 · 협업형 작업

⚡ Fast Mode: 작고 명확한 작업



변수명 변경, 간단한 명령 실행, 작은 코드 수정



넓은 범위, 애매한 요구사항에서는 품질 저하 위험

⚡ 핵심 기준: 작아서 계획 비용이 더 큰 작업



Artifact Review Policy: 계획 검토 시점

Request Review

계획 제시 후 검토 대기

Always Proceed

검토 요청 없이 실행 단계 진행

 학습 권장값: Request Review — 계획 읽기와 방향 확인

Terminal Auto Execution: 명령 실행 정책

Request Review

기본 자동 실행 중지, 필요 시 승인

Allow / Deny List

안전 명령 허용, 위험 명령 차단

 확인 기준: “왜 이 명령이 필요한가?”

📁 File Access: Workspace 안팎의 경계

✅ DEFAULT

Workspace 내부와 Antigravity 관련 폴더

현재 workspace와 Antigravity 애플리케이션
데이터 범위



⚠️ OPTIONAL

Non-workspace file access

workspace 바깥 파일 접근 가능. 민감 정보 노출 위험 존재



Agent Permissions: 자동화 안전 레일

 **Allow**

묻지 않고 자동 승인

 **Ask**

실행 전 사용자에게 확인

 **Deny**

즉시 차단

Permission Resource: action(target)

action (target)

 **action**

행동 종류

 **target**

적용 대상 또는 패턴

Permission Actions: 5가지 유형

 **command**

터미널 명령 실행 제어

 **read_file**

파일·디렉터리 읽기 제어

 **write_file**

파일·디렉터리 쓰기 제어

 **read_url**

도메인 단위 웹 읽기 제어

 **mcp**

MCP 서버·도구 사용 제어

 **Permissions의 의미: Agent에게 맡길 수 있는 작업 범위의 명확화**

Permission Examples: 실무형 패턴

✅ Git 명령 자동 허용

command(git)을 Allow에 추가

🚫 위험 명령 차단

command(rm), command(sudo)를 Deny에 추가

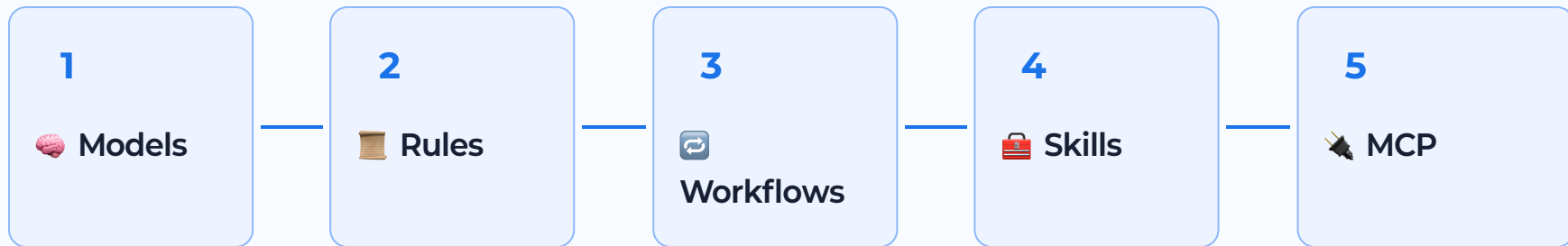
👁️ 모든 명령 확인

command(*)를 Ask에 추가

🔌 MCP 도구 제한

mcp(server/tool) 기준 서버·도구 단위 제어

📍 운영 확장 맵: Models → Rules → Workflows → Skills → MCP



🎯 목표: Agent의 판단·반복·도구 연결을 프로젝트 운영 방식으로 고정



Model Selection: 작업 성격별 선택



DEEP REASONING

설계·리서치·복잡한 변경

- 큰 코드베이스 분석
- 여러 파일 영향도 판단
- 계획과 검토가 중요한 작업



FAST MODEL

작고 빠른 편집

- 짧은 수정
- 간단한 명령 생성
- 반복적인 작은 작업



Model Behavior: 대화 단위 유지

01



드롭다운 선택

대화 입력창 아래 모델 선택

02



대화 유지

한 Agent 대화 안에서 선택값 유지

03



다음 요청 적용

실행 중 변경은 다음 사용자 요청부터 적용



실행 중단 시 예외적으로 흐름 전환 가능

🤝 Helper Models: 보조 모델의 자동 역할

🎨 이미지·목업

UI 목업, 웹페이지 이미지, 다이어그램 생성 보조

🌐 브라우저 조작

클릭, 스크롤, 입력 등 Browser Subagent 작업 보조

🧠 문맥 요약

긴 작업 흐름의 체크포인트와 요약 처리

🔍 Semantic Search

코드베이스 검색과 관련 문맥 발견 보조

핵심 감각: 사용자가 직접 고르는 모델과 내부 보조 모델의 역할 분리

Browser Agent: 코드 밖 작업 표면

 대시보드 확인

 SCM 작업

 웹 UI 테스트

 브라우저 기반
개발

 리서치 보고서

 의미: 개발자가 브라우저에서 하던 확인·조작 일부를 Agent 루프에 포함

🌐 Browser Agent Flow: 화면 읽기 → 조작 → 증거



Rules: Agent가 따를 작업 규칙

 **프로젝트 스타일, 구조, 금지 사항, 작업 방식을
Markdown 규칙으로 고정**

전역 수준과 workspace 수준에서 Agent의 기본 행동 기준 제공

 비유: “매번 말하는 지시”를 “항상 참고할 운영 문서”로 전환

📏 Rules Scope: Global vs Workspace

🌐 GLOBAL

모든 workspace 공통

`~/.gemini/GEMINI.md`



📁 WORKSPACE

프로젝트 전용 규칙

`.agents/rules` 중심, 기존 `.agent/rules` 호환

🎯 팀 프로젝트 규칙은 workspace rule로 관리

Rule Creation Flow: Customizations 패널

01

… 메뉴

Editor의 Agent 패널 상단
드롭다운

02

 Customizations

Rules 패널 이동

03

+ Global /
Workspace

전역 또는 workspace 규칙 생성

⚙️ Rule Activation: 4가지 적용 방식

👉 Manual

Agent 입력창에서 직접 호출

∞ Always On

항상 적용되는 기본 규칙

🧠 Model Decision

설명을 보고 모델이 적용 판단

🧩 Glob

`src/**/*.ts` 같은 파일 패턴 기준



실습 기준: 처음에는 Manual 또는 Glob로 범위를 좁혀 안전하게 확인

Rules Example: 팀 코드 스타일 고정

예시 위치

```
.agents/rules/  
└── frontend-style.md
```

Always On

Glob: src/**/*.tsx

12,000자 제한

Frontend Style

- React component는 함수형으로 작성
- 복잡한 상태는 custom hook으로 분리
- 변경 후 pnpm run build 실행
- UI 변경은 브라우저 캡처로 검증

🔗 @ Mentions: 규칙 안의 파일 참조



Rule 파일

@filename 형식 참조



상대 경로

Rules 파일 위치 기준 해석

또는



절대 경로

실제 절대 경로 우선 해석



Rules Lab: 활성화 방식 비교



Manual

특정 요청에서만 규칙 호출



Always On

팀 공통 품질 기준에 적합



Glob

파일 유형별 convention에 적합



확인 포인트: Agent 답변이 규칙을 실제로 반영하는지 diff와 설명으로 검토

Workflows: 반복 절차의 slash command화

 배포, PR 대응, 릴리즈 체크리스트처럼 반복되는 절차를
Markdown workflow로 저장

Agent 입력창에서 `/workflow-name` 형태로 실행

Rules vs Workflows: 규칙과 절차의 차이

RULES

항상 참고할 행동 기준

코드 스타일, 프로젝트 구조, 금지 사항, 검증 원칙

지속 문맥

제약

판단 기준

VS

WORKFLOWS

특정 작업의 실행 순서

배포 절차, 리뷰 대응, 릴리즈 점검, 반복 업무

단계

체크리스트

slash command

Workflow Creation: Global / Workspace

01

… 메뉴

Agent 패널의 Customizations
패널

02

 **Workflows**

Workflows 패널 이동

03

+ 생성

Global 또는 Workspace
workflow 작성



Workflow 파일도 12,000자 제한 기준으로 간결하게 구성



Workflow Anatomy: 좋은 절차 문서

제목

어떤 작업을 위한 절차인지 명확한 이름

설명

언제 실행할 workflow인지 사용 조건

단계

Agent가 따라야 할 순서와 중간 확인 지점

산출물

마지막에 남겨야 할 결과 보고 형식

Workflow Example: PR 코멘트 대응

PR Review Response

1. 미해결 리뷰 코멘트 수집
2. 코멘트별 요구사항 분류
3. 관련 파일 최소 범위 수정
4. 테스트 또는 빌드 실행
5. 변경 요약과 남은 리스크 보고

 실행 예시: `/pr-review-response`



Workflow Lab: 반복 절차 만들기



작업 선택

자주 반복되는 릴리즈 점검 또는 PR 대응
선정



단계 정리

조사, 수정, 검증, 보고 순서로 작성



실행 확인

slash command로 실행 후 산출물 검토

Skills: 재사용 가능한 작업 설명서

 **특정 작업 유형을 더 잘 수행하도록 instructions, best practices, scripts, resources를 묶은 패키지**

Agent가 skill 이름과 description을 보고 필요한 지침을 단계적으로 읽는 구조

Skill Anatomy: 폴더와 SKILL.md

기본 구조

```
.agents/skills/  
└── my-skill/  
    ├── SKILL.md  
    ├── scripts/  
    ├── examples/  
    └── resources/
```

instructions

작업 처리 방법

practices

권장 기준과 convention

scripts

반복 자동화

resources

템플릿과 참고 자료

Skill Scope: Workspace vs Global

WORKSPACE

프로젝트 전용 skill

```
<workspace-  
root>/.agents/skills/<skill-  
folder>/
```



GLOBAL

개인 공통 skill

```
~/.gemini/antigravity/skills/<skill-folder>/
```

SKILL.md Frontmatter: description 중심

필수 판단 단서

Agent는 skill 목록의 name과 description을 보고 현재 작업과의 관련성을 먼저 판단

name: optional

description: required

name: code-review

description: Reviews code changes for bugs, style issues, and best practices. Use when reviewing PRs or checking code quality.

Code Review Skill

Progressive Disclosure: 필요한 만큼 읽기



Skill Best Practices: 작고 명확한 단위

좁은 범위

하나의 skill이 너무 많은 일을 담당하지 않도록 분리

구체적 description

사용 시점과 키워드를 명확하게 작성

scripts는 블랙박스

소스 전체보다 --help와 실행 결과 중심

decision tree

복잡한 skill에는 분기 기준 포함

Skill Example: Code Review Skill

1. Correctness

요구사항을 실제로 만족하는지 확인

2. Edge cases

오류 조건과 경계 조건 확인

3. Style / Performance

프로젝트 convention과 비효율 확인



Slidev Skill Lab: 작은 장표 생성



Goal

README의 핵심 흐름을 3장짜리
Slidev 장표로 정리



Constraints

한 장당 메시지 하나, 카드 배경 불투명,
build/export 통과



Evidence

PNG 렌더링과 폰트 적용 확인 결과 보고

MCP: 외부 context와 tool 연결

 **Model Context Protocol은 Agent가 외부 시스템의 context와 custom tool을 사용할 수 있게 하는 연결 방식**

GitHub, Notion, Linear, Figma, 데이터베이스, 클라우드 리소스를 작업 맥락으로 확장

MCP Value: Context Resources + Custom Tools

CONTEXT RESOURCES

읽기 가능한 외부 맥락

DB schema, 빌드 로그, GitHub issue, 디자인 자료 등

읽기

판단 근거

context



CUSTOM TOOLS

외부 시스템 작업 실행

issue 생성, 검색, 데이터 조회, 자동화 작업 등

행동

API

tool

MCP Store Flow: 설치형 연결

01

MCP Store

Agent panel 상단 메뉴에서 MCP Store 열기

02

Install

지원 서버 선택 후 설치

03

Auth

필요한 계정 인증 진행



Custom MCP Config: 직접 서버 연결



설정 위치

~/gemini/antigravity/
└─ mcp_config.json

[Manage MCP Servers](#)

[View raw config](#)

```
{
  "mcpServers": {
    "serverName": {
      "command": "path/to/executable",
      "args": ["--arg1", "value1"],
      "env": { "API_KEY": "..." }
    }
  }
}
```

🔒 MCP Auth: 인증 방식 3가지



Google Credentials

ADC 기반 인증 연결



OAuth

Authenticate 버튼과 브라우저
인증 흐름



Custom Headers

API key 또는 bearer token 직접
지정



MCP Permission: 연결 후에도 안전장치

mcp (server/tool)

✅ **Allow**

안전하고 반복적인 도구 자동 허용

🛑 **Ask / Deny**

민감 데이터 접근이나 쓰기 작업 통제

🌐 MCP Ecosystem: 연결 후보

🐙 GitHub

🔥 Firebase

📋 Linear

📝 Notion

🎨 Figma

📊 BigQuery

🐘 Cloud SQL

💳 Stripe

🧭 **선택 기준: Agent가 작업 판단에 실제로 필요한 context 또는 tool 여부**



Editor: 익숙한 IDE + AI 작업 표면



VS Code 계열 편집 경험

파일, 탭, 터미널, source control 중심의
익숙한 개발 화면



Tab

코드 제안과 이동



Command

inline code·terminal command 생성



Agent Side Panel

큰 작업 위임

✅ Editor Review: 변경 확인과 소스 컨트롤

1. 📄 Review Changes

Agent가 만든 diff와 변경 범위 확인

2. ✅ 검증 결과

테스트, 빌드, 브라우저 확인 결과 연결

3. 🔄 Source Control

커밋 전 변경 의도와 파일 범위 점검

Open VSX: 에디터 경험 보강



Syntax Highlighting

언어별 가독성 향상



Source Control

Git 관련 확장과 워크플로 보강



Test Tools

프로젝트별 테스트 실행 경험 강화



Productivity

기존 VS Code 계열 extension 활용



Tab: Editor 안의 빠른 AI 보조



Supercomplete

파일 전체 맥락 기반 코드 제안



Tab-to-Jump

다음 편집 위치로 이동



Tab-to-Import

단어 완성과 import 추가

✨ Supercomplete: 한 위치를 넘어선 제안

일반 완성

현재 커서 주변

가까운 줄의 코드 완성 중심



✨ SUPERCOMPLETE

파일 전체 맥락

변수명 동시 변경, 관련 함수 갱신 같은 넓은 제안

Tab Navigation: Jump와 Import



Tab-to-Jump

다음 편집 위치 제안



Tab-to-Import

필요한 import 자동 제안



흐름 유지

마우스 이동과 검색 비용 감소

⚙️ Tab Settings: 속도와 범위 조절



on/off

Autocomplete,
Supercomplete, Jump,
Import 개별 제어



속도

Slow, Default, Fast 제안 속도



Highlight

삽입 텍스트 표시 여부



Clipboard

clipboard context 사용 여부

⚡ Command: 자연어 → 코드·터미널 명령





Command in Editor: inline code 생성

요청 예시

Create a React component
for a login form.

적합 작업

boilerplate code, 함수 refactor,
문서 작성

검토 기준

현재 파일 맥락, 변경 범위, 스타일
규칙 반영

Command in Terminal: shell 문법 보조

요청 예시

Find all processes listening on port 3000 and kill them.

강점

복잡한 shell command를
자연어에서 생성

주의

위험 명령은 Permissions와 사용자
검토로 제어

Hands-on Lab: Release Readiness MCP

 **열린 파일에 없는 릴리즈 맥락을 MCP로 읽고, 릴리즈 진행 여부를 근거 기반으로 판단**

문제 → 연결 방식 → 실제 조작 → 결과 확인 → 의미 정리 흐름

 실습 결과물: 조건부 진행/보류 판단, 위험 버그 요약, 팀별 액션



Lab Setup: 데모 문서와 MCP 범위



실습 폴더

~/antigravity-mcp-lab

개인 문서나 홈 디렉터리 전체를 노출하지 않고 실습 폴더만
읽기

```
bash exercise-script/01-create-lab-files.sh
```

생성 파일

- release-notes.md
- decision-log.md
- bug-report.md
- support-ticket.md

Lab Run: MCP 근거 기반 질문

1. Context Check

접근 가능한 파일 목록과 판단 근거 요약

2. Release Decision

진행 / 보류 / 조건부 진행 중 결론

3. Risk Drill-down

가장 위험한 버그와 검증 체크리스트 정리

Lab Prompt: Release Decision

현재 릴리즈를 진행해도 되는지 판단해줘.

반드시 releaseReadiness MCP에서 읽은 파일들을 근거로 사용하고,
릴리즈 진행 / 보류 / 조건부 진행 중 하나로 결론을 내려줘.

마지막에는 오늘 바로 처리할 작업 3개를 우선순위로 제안해줘.

 기대 근거: BUG-142 · orderId · rollback 기준 · 쿠폰 결제



Debrief: 실습의 의미

1



Context 확장

열린 파일 밖의 근거를 안전하게 연결

2



Evidence 중심

일반론보다 파일 기반 판단과 출처 확인

3



범위 통제

MCP permission과 폴더 범위로 안전성 확보

Firebase Studio Migration: 이동 전 점검

프로젝트 구조

기존 앱, 설정 파일, 빌드 스크립트 위치 확인

권한·비밀값

환경 변수와 인증 정보 노출 범위 점검

검증 루프

마이그레이션 후 build, run, browser
확인 기준 준비

 이동의 핵심: 파일 복사보다 Agent가 이해할 수 있는 workspace와 규칙 구성



이
0

Migration Prompt Pattern: 현황 → 계획 → 검



현황 파악

구조, 의존성, 실행 방법



전환 계획

작업 순서와 리스크



검증 기준

빌드, 테스트, 화면 확인



추천: Planning Mode와 Request Review로 전환 계획 먼저 검토

✓ Checkpoint: 전체 흐름 정리

1

🖥️ 작업 표면

Manager, Editor, Browser의 역할
구분

2

🤖 Agent 운영

모드, 산출물, 권한 기반으로 작업 위임

3

🔄 반복 확장

Rules, Workflows, Skills, MCP로
작업 방식 재사용

🧭 핵심 문장: Agentic Coding은 “잘 시키기”보다 “작업 단위 설계와 근거 기반 검토”

- Source: [Get Started](#)
- Source: [Agent](#)
- Source: [Models](#)
- Source: [Agent Modes / Settings](#)
- Source: [Agent Permissions](#)
- Source: [Rules / Workflows](#)

- Source: [Skills](#)
- Source: [MCP](#)
- Source: [Editor](#)
- Source: [Tab](#)
- Source: [Command](#)
- Source: [Firebase Studio Migration](#)